Reproducing LiTS with DeepSeek

The effect of maximum search depth on a small MATH500 benchmark

FTEC5660 - Agentic AI for Business and FinTech
Name: SANG Yuchen Student ID: 1155262204

Reproducibility report | 4 November 2026 | Individual assignment

1. Objective and scope

This study runs the authors' LiTS implementation of MCTS mathematical reasoning using an accessible DeepSeek API model. It reproduces a runnable agentic workflow, then changes one experimental factor: the maximum number of steps on a search-tree path. The measured outcomes are accuracy, token usage, model-call count, wall time and observed tree depth.

The numerical reference is LiTS paper v2, Table 6: ReST (MCTS) reports 37% on 100 numerical-answer MATH500 questions using Llama3-8B and the paper's reward setup. Our DeepSeek model, API-based GenerativePRM, current repository snapshot, smaller question prefix and bounded-search settings differ. Our numbers therefore do not constitute a strict replication of that 37% result or a causal estimate of the benefit of replacing Llama3 with DeepSeek.

Main measured findings

The default-depth group (10 steps) achieved 85.6 +/- 5.1% accuracy across three runs. The highest mean accuracy was 90.0%, shared by settings [1, 3]; this descriptive comparison is specific to these 30 questions. The lowest measured token use was at depth 1. All nine formal runs completed, covering 270 question-runs on 30 distinct questions, with 4,425 model calls in total.

What was reproduced

The fixed local dataset loaded into the repository's registered MATH500 task; its ConcatPolicy, ConcatTransition and GenerativePRM executed real MCTS searches; saved terminal answers were grouped through the repository's voting workflow and checked using the author's deterministic numeric checker. The full trees, configurations and per-call usage provide inspectable evidence.

2. Setup and controlled experimental design

Item	Fixed setting
Source	25e94d862cb4514c22d9a80a635e392461769df7
Environment	Windows; isolated CPython 3.12.12; exact API-path dependency lock
Dataset	xinzhel/math500-float; test.jsonl; 316 numeric rows; first 30 rows in original order
Dataset revision	d27dc61b1123c2fdc4eea855c802be87dfa9d536
Model / endpoint	deepseek-flash / https://api.deepseek.com; internal thinking disabled
Search	MCTS; 10 iterations; branching factor 3; rollout length 2; simulation branching 1
Depth experiment	max_steps = 1, 3, 10; forced depth limit enabled in every group
Scoring and termination	GenerativePRM: correctness samples 2, usefulness samples 1; termination samples 3
Generation / repeats	Policy output cap 512 tokens; unchanged role-specific decoding; local seeds 11, 22, 33
Execution	Initial nine-process batch; one full retry alone; same source fingerprints; SDK retries off; timeout 90 s

Temperature is not one universal scalar in LiTS: generation, scoring, termination checks and voting use their existing role-specific values. Every outgoing request records its temperature, top_p, max_tokens and thinking payload. Recorded requests include top_p values 0.95 and 1.0 rather than one global value. Local Python/NumPy seeds are set; no claim is made that the remote model is seed-deterministic.

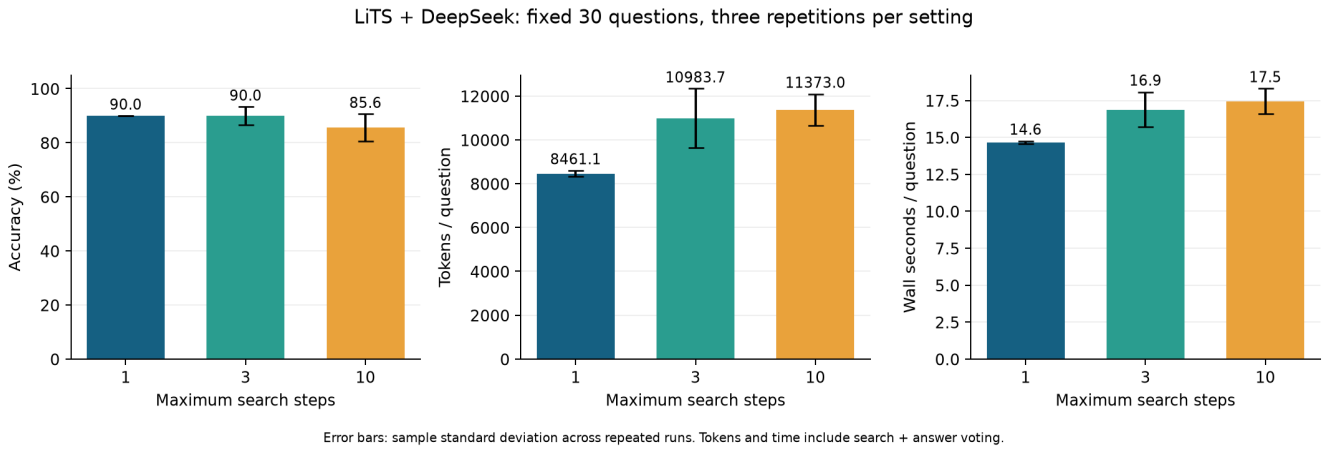
Pilot and design freeze

A one-question smoke run verified end-to-end execution. A separate five-question pilot scored 4/5 and reached maximum tree depths [1, 1, 1, 3, 1]. Before running any formal trial, the planned grid was changed from [5, 10, 15] to [1, 3, 10], so the experiment includes a short binding budget and the default-depth comparison. This decision and its earlier grid are recorded in the plan and Git history. The pilot overlaps with the formal prefix; this is an exploratory study, not an untouched held-out evaluation.

3. Formal results

Each row below is the mean and sample standard deviation across three completed runs, each using the same 30 questions. Token usage and wall time include both search and final answer voting. Correctness is numeric checking after extraction; failed or unfinished setup runs are not included in these denominators.

Max steps	Accuracy (%)	Calls / question	Tokens / question	Seconds / question
1	90.0 +/- 0.0	15.1 +/- 0.2	8461 +/- 140	14.6 +/- 0.1
3	90.0 +/- 3.3	17.1 +/- 1.1	10984 +/- 1353	16.9 +/- 1.2
10	85.6 +/- 5.1	16.9 +/- 0.4	11373 +/- 719	17.5 +/- 0.8



Observed depth

Configured cap	Mean maximum depth per question	Deepest observed node
1	1.00 +/- 0.00	1
3	1.17 +/- 0.03	3
10	1.16 +/- 0.04	4

All saved tree depths were checked against their configured cap. Depths are reconstructed from full-tree child links, rather than inferred from model-call count. The number of calls can greatly exceed the path depth because each expansion samples alternatives and evaluates reasoning steps.

These error bars summarize repeat-to-repeat variation on one fixed small dataset. They are not population confidence intervals. Hosted API load and concurrency affect wall time. The initial batch ran nine processes; the depth-10/seed-11 replacement ran alone after the file-write failure. Timing is descriptive and not a controlled latency comparison across identical load conditions.

4. Trial-level evidence

Cap	Local seed	Correct / 30	Accuracy	Search calls	Vote calls	Max depth
1	11	27/30	90.0%	429	29	1
1	22	27/30	90.0%	429	29	1
1	33	27/30	90.0%	418	29	1
3	11	28/30	93.3%	509	30	3
3	22	27/30	90.0%	491	30	3
3	33	26/30	86.7%	446	31	2
10	11	24/30	80.0%	489	30	4
10	22	26/30	86.7%	466	30	3
10	33	27/30	90.0%	480	30	3

Metric definitions

Accuracy: correct predictions divided by all 30 planned questions in a completed run. An empty or missing final answer is incorrect. A run interrupted by an API, resource or artifact error is marked incomplete and retained separately, rather than silently shrinking the denominator.

Tokens: provider-reported prompt_tokens plus completion_tokens, aggregated from api_calls.jsonl. Search and voting totals remain separately available. Input-cache details are preserved in the raw usage fields. Provider errors are recorded and abort the run instead of becoming model mistakes.

Time: elapsed time around the search command plus its evaluation command, divided by 30. Depth: the largest parent-child distance from the root for each question. Variability: sample SD and sample variance over the three repeated runs, with variance included in the machine-readable summary.

Post-hoc scoring audit

A manual audit identified 16 format-only false negatives across the 270 formal question-runs in three known tasks. Examples are a 72-degree rotation predicted as '72 degrees' or '72°' against reference '72', profit '$78' against '78', and side length '12 cm' against '12'. The questions explicitly request these units. The original checker does not consistently strip units or currency from predictions.

The primary accuracies above remain unchanged. This diagnostic was discovered after execution and is not a replacement endpoint or a basis for selecting runs. It shows that the reported scores measure output formatting as well as numerical problem solving; small accuracy differences should not automatically be attributed to reasoning quality.

Interpreting the modification

Changing max_steps affects LiTS's external search budget and its last-step instruction. It does not set the number of arithmetic operations or the model's internal reasoning tokens. In the pilot, the model often returned a complete derivation within one generated action. Consequently, a shallow tree can still contain substantial reasoning, and deeper search can add scoring/call cost without guaranteeing a better final answer.

No result was selected by looking at the reference answer: references are reserved for checking. The repository's generative reward model and answer grouping are themselves model-based, however, and can introduce errors or preferences.

5. Debug diary and adaptations

Environment: the machine initially had Python 3.14 and no Git on PATH. A workspace-local Git installation and separate Python 3.12.12 environment were prepared. Only the API mathematics path was installed; local GPU, SQL, geospatial and memory functionality is outside the tested scope. An unused torch import in config.py was removed.

Endpoint compatibility: the wrapper originally sent vLLM-specific thinking options to all custom endpoints. It now sends DeepSeek's thinking.type field. DeepSeek's stable tool endpoint omits the beta-only strict flag while preserving the schema and LiTS's local validation. The same adapter is used in every formal group.

Windows artifact failure: the first live attempt completed 11 requests but failed while writing a superscript character under cp950. The failed run was retained and excluded from accuracy statistics. The launcher now uses UTF-8. A subsequent restart failure occurred before any API request; subprocess waiting was corrected. The successful smoke and all formal trials use the repaired path. One initial depth-10/seed-11 formal attempt failed during an atomic completion-marker replacement (WinError 5) after writing the search artifacts. That incomplete attempt was preserved, excluded from the main table, and the entire configuration was rerun once without changing the experimental source. The other eight groups were retained. Concurrent Windows state-file monitoring is a possible contributor, not a proven cause.

Version control and evidence: source and dataset revisions are fixed; the dataset hash is checked on load and its byte representation is protected from Git newline conversion. Every run writes source hashes, dependency versions, parameters, trees, per-call usage and a completion status. The .env key is ignored by Git and excluded from all submission artifacts.

6. Conclusions and limitations

The API-backed LiTS workflow is runnable under the recorded conditions. At default depth 10 its measured accuracy was 85.6 +/- 5.1%. The step-budget experiment demonstrates how to measure the accuracy/resource tradeoff using one isolated configuration change; the tables report the measured effect without treating the paper's different-model result as an equivalent baseline.

The main limits are the small, ordered 30-question subset; three repetitions on the same tasks; pilot overlap; an evolving remote model alias; a different reward-model setup from the paper; model-based answer grouping; and strict numeric normalization that may not recognize every equivalent expression. Results should not be generalized to all MATH500 questions or used to claim that DeepSeek universally outperforms the original model.

Reproduction commands

See assignment/README_CN.md. Rebuild the Python environment from requirements-api.lock.txt; prepare or verify the fixed data; place the key only in .env; run the preflight; then execute assignment.study with the saved phase, maximum step count and local seed. Raw run artifacts remain locally available; selected formal evidence is included in the submission package.

References

LiTS paper v2 (Table 6 and setup): https://arxiv.org/html/2603.00631v2

Pinned LiTS source: https://github.com/xinzhel/lits-llm/tree/25e94d862cb4514c22d9a80a635e392461769df7

Pinned numeric dataset: https://huggingface.co/datasets/xinzhel/math500-float/tree/d27dc61b1123c2fdc4eea855c802be87dfa9d536

DeepSeek thinking control: https://api-docs.deepseek.com/guides/thinking_mode/

DeepSeek tool API: https://api-docs.deepseek.com/guides/tool_calls/